

unprecedented / one piece / unveiled / profound

- open source package: VLLM / SGlang / Tensor-RT (Nvidia) / TGI

def review - transformer?

Transformer Architecture: Flops for a feedward pass: $6 * (B * D) * F$

def review - of arithmetic intensity:

setup: multiply $X (B * D)$ and $W (D * F)$ matrices

intuition: B is Batch size, D is the hidden dimension, F is up-projection dimension in MLP.

Let's do Flops and memory read/write accounting for matrix multiplication $(X * W) \Rightarrow$

$\Rightarrow \text{Flops} = 2 * B * D * F$
(multiply-add)
 $\Rightarrow \text{Bytes-transferred} = 2 * B * D + 2 * D * F + 2 * B * F$
(CFPb: 2 Bytes per FP16 element)

$\Rightarrow$ Arithmetic intensity is how much compute we do per byte transferred

$\text{intensity} = (\text{Flops} / \text{bytes-transferred}) \cdot \text{simplify}(1)$

$= \cancel{B} \cdot \text{intensity} \cdot \text{subs}(D, C * B) \cdot \text{subs}(F, C * B) \cdot \text{limit}(C, \infty) \cdot \text{simplify}(C)$
 $= B$

demo
Accelerator intensity of H100:

$\text{flops-per-sec} = 9.89e^{12}$

$\text{memory-HBW} = 3.35e^{12}$

$\text{Accelerator-intensity} = \text{flops-per-second} / \text{memory-bandwidth}$

$\text{round}(\text{Accelerator-intensity}) = 295$

if computation intensity > accelerator intensity, compute-limited (good)

if computation intensity < accelerator intensity, memory-limited (bad)

$\Rightarrow$ compute-limited if $B < 295$

for generation task, memory-limited cases are often

(memory bound)

e.g., extreme case: $B=1$: read 1x1 matrix, perform only 2DFlops

def Arithmetic_intensity_of_inference():

naive: prefill-decode-process: $\downarrow$ Auto-regressive

$\hookrightarrow$ to generate each token, feed history into transformer

complexity: generating T tokens require $O(T^3)$ Flops (one feedforward pass is $O(T^{1/2})$)

$\Rightarrow$ observation: a lot of work can be shared across prefixes

Solution: store K-V cache in HBM

$\hookrightarrow$ for every sequence (B) token (S), layer (L), head (K), store an H-dimensional vector

two stages of inference:

1: pre-fill: given a prompt, encode into vectors (parallelize like in pre-training)

2: generation (decode): generate new responses tokens (sequential)

Examples: compute the FLOPs and memory IO for both the MLP and Attention

S: num of token we condition on

(layers)

T: num of token we generate to

$\Rightarrow$ prefill: $T=S$

generation: $T=1$

$N/K = G$: 99 heads per kv

heads=

B: Batch L: num of layers S: sequence length V: vocab N: Num of query heads

D: d-model, embed-dim, F: MLP hidden dim, H: attention hidden dim K: Num of key/value heads